Learning Representations for Graph Retrieval

Omm Arindam
24110229@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Rajput Priyanshu
priyanshu.raj@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Ramji Purwar
ramji.purwar@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Abstract

We propose a scalable graph retrieval pipeline that combines Graph Neural Network (GNN) based graph embeddings with Locality Sensitive Hashing (LSH) to enable approximate nearest neighbor search over large graph corpora. Given a corpus of graphs and a query graph, naive pairwise comparison is computationally infeasible due to the NP-hard nature of graph isomorphism and subgraph matching. We aim to design graph embeddings that capture structural and semantic properties of graphs using a Graph Isomorphism Network (GIN) trained with triplet loss, and develop an LSH scheme over these embeddings for efficient retrieval without exhaustive pairwise comparison. We evaluate on standard graph benchmarks including MUTAG, PROTEINS, IMDB-B, Reddit-Binary, and AIDS using Precision@k, Recall@k, MAP, query time, and approximation quality metrics.

CCS Concepts

• **Mathematics of computing** → **Graph algorithms**; • **Computing methodologies** → *Graph-based machine learning*; • **Information systems** → *Nearest-neighbor search.*

Keywords

graph retrieval, graph embeddings, locality sensitive hashing, graph neural networks, approximate nearest neighbor search

1 Motivation

Graphs are a powerful and expressive data structure used to represent relational information across a wide range of domains including social networks, molecular chemistry, knowledge bases, and program analysis. As graph-structured datasets grow in size, the problem of *graph retrieval* — finding graphs in a large corpus that are most similar to a given query graph — has become increasingly important.

A naive approach compares the query against every graph in the corpus, which is prohibitively expensive. Even polynomial-time graph similarity measures such as graph edit distance (GED) or maximum common subgraph (MCS) are NP-hard in the general case, making exhaustive search completely infeasible for large corpora. For example, comparing a query molecule against a database of millions of chemical compounds using GED is computationally intractable without approximation.

This limitation motivates the use of dense vector representations that capture graph structure and semantics. By embedding both queries and corpus graphs into a shared vector space, we enable retrieval based on structural similarity rather than exact isomorphism testing.

2 Goal

The goal of this project is to design and implement a scalable graph retrieval system that retrieves the most structurally and semantically similar graphs from a large corpus for a given query graph, without exhaustive pairwise comparison.

The system is designed as an end-to-end pipeline consisting of graph embedding generation, contrastive training, LSH-based indexing, and approximate nearest neighbor retrieval.

2.1 Embedding Generation

In traditional graph retrieval, similarity is computed directly via graph edit distance or subgraph matching — both NP-hard operations. Such approaches do not scale to large corpora.

To address this, we encode both corpus graphs and query graphs into fixed-dimensional dense vector representations using a Graph Neural Network. We use the **Graph Isomorphism Network (GIN)** [5], a GNN architecture provably as expressive as the Weisfeiler-Leman graph isomorphism test, making it a strong baseline for structure-sensitive embeddings.

GIN operates via iterative message passing:

$$h_v^{(\ell)} = \text{UPDATE}\Big(h_v^{(\ell-1)},\ \text{AGGREGATE}\Big(\big\{h_u^{(\ell-1)} : u \in \mathcal{N}(v)\big\}\Big)\Big) \tag{1}$$

After L layers, a global readout function (sum or mean pooling) produces the graph-level embedding:

$$z_G = \textsc{Readout}\Big(\big\{h_v^{(L)} : v \in V\big\}\Big) \tag{2}$$

Node Feature Initialization. Datasets vary in whether they provide node features. We handle this as an explicit design decision:

- **Datasets with node features** (MUTAG, PROTEINS, AIDS): Atom type, charge, or secondary-structure labels are used directly as input node features, one-hot encoded where categorical.
- **Datasets without node features** (IMDB-B, Reddit-Binary): We use degree-based one-hot encoding as the input node feature, following the original GIN paper [5]. This preserves structural inductive bias while avoiding the uniform bias of constant features or the noise introduced by random initialization. We use a degree vocabulary capped at the 95th percentile of degree values per dataset to handle outlier high-degree nodes.

These dense embeddings enable the system to capture structural similarity between graphs even when they are not isomorphic.

2.2 Similarity Measure

Graph similarity in the embedding space is measured using Euclidean (ℓ_2) distance:

$$\text{dist}(z_Q, z_G) = \|z_Q - z_G\|_2 \tag{3}$$

A smaller distance indicates greater structural similarity between query graph Q and corpus graph G. The embedding network is trained explicitly so that similar graphs are close and dissimilar graphs are far apart in this space.

2.3 Indexing and Retrieval

Once graph embeddings are generated, they are indexed to enable efficient retrieval using Locality Sensitive Hashing (LSH) with random projection [2]. Each hash function takes the form:

$$h_{a,b}(z) = \left\lfloor \frac{a \cdot z + b}{w} \right\rfloor \tag{4}$$

where $a \sim \mathcal{N}(0, I_d)$, $b \sim \text{Uniform}[0, w]$, and w is a bandwidth parameter. Multiple hash tables are constructed using independent hash functions to increase recall.

LSH Parameter Selection. The bandwidth w and number of hash tables L directly control the precision-recall trade-off:

- **Bandwidth w:** We use the theoretical collision probability curve [2] to initialize w such that the collision probability at the target similarity threshold is approximately 0.9, then refine via a small grid search over $\{0.5, 1.0, 2.0, 4.0\}$ on a 10% held-out corpus split, selecting the value that maximizes Recall@10.
- **Number of hash tables L:** We set $L = 10$ as a default and ablate over $L \in \{5, 10, 20\}$ to measure the recall-vs-memory trade-off.
- **Hash functions per table K:** Set to $K = 4$, following the analysis in [3], which shows $K = O(\log n)$ functions per table gives good bucket granularity for corpus sizes in the hundreds to thousands.

At query time, the query graph Q is embedded to z_Q, hashed into each table, and candidate graphs are retrieved from matching buckets and re-ranked by exact embedding distance, yielding the final top-k results.

2.4 Why Approximate Nearest Neighbor (ANN)?

Exact nearest neighbor search requires comparing the query embedding against every graph embedding in the corpus, giving linear time complexity in corpus size. LSH significantly reduces search time by hashing similar vectors into the same buckets with high probability, enabling sublinear-time retrieval while achieving high accuracy. This trade-off enables real-time graph retrieval over large corpora.

2.5 Research Objective

The primary research question of this project is: *To what extent does GNN-based embedding retrieval improve over naive pairwise comparison baselines, and how does LSH-based approximate retrieval compare to exact nearest neighbor search under identical embedding conditions?*

3 Query Flow

At query time, the system processes a user-supplied query graph through the following pipeline:

(1) **Query Input:** The user provides a query graph $Q = (V, E)$, possibly with node and edge features.
(2) **Query Encoding:** Q is passed through the trained GIN model to produce a fixed-dimensional embedding z_Q. The same weights used to embed corpus graphs are used here, ensuring both lie in the same vector space.
(3) **LSH Lookup:** z_Q is hashed into each of the LSH hash tables. All graphs whose buckets collide with the query are retrieved as candidates.
(4) **Re-ranking:** Candidate graphs are re-ranked by their exact Euclidean distance to z_Q.
(5) **Output:** The top-k graphs are returned as the final retrieval results, ranked by embedding similarity.

4 Evaluation

4.1 Datasets

We evaluate on the following graph benchmark datasets:

- **MUTAG**: Chemical compounds (mutagenic molecules) as graphs; nodes represent atoms and edges represent chemical bonds.
- **PROTEINS**: Protein structures as graphs, where nodes correspond to secondary structure elements.
- **IMDB-Binary (IMDB-B)**: Actor/actress ego-networks in movie collaborations; nodes carry no feature labels, testing purely structural retrieval.
- **Reddit-Binary**: Large graphs representing online discussion threads; a scalability stress test.
- **AIDS**: Chemical compound graphs for antiviral activity screening.

Together these datasets provide a comprehensive evaluation across molecular, biological, social, and large-scale domains.

4.2 Metrics

The definition of ground-truth relevance directly affects metric validity. We use a two-tier oracle strategy to strictly separate training labels from evaluation ground truth (see Section 5):

- *Small datasets* (MUTAG, AIDS, PROTEINS; graphs with ≤ 30 nodes): R_q is defined using exact GED computed via the A*-based GEDLib library.
- *Larger datasets* (IMDB-B, Reddit-Binary): R_q is defined using high-quality beam search GED (beam width $B = 20$), held entirely separate from the training oracle ($B = 5$).

Let R_q denote the ground-truth relevant set for query q and $L_q^{(k)}$ the top-k retrieved graphs. We report:

$$\text{Precision@}k = \frac{1}{|Q|} \sum_q \frac{|R_q \cap L_q^{(k)}|}{k} \tag{5}$$

$$\text{Recall@}k = \frac{1}{|Q|} \sum_q \frac{|R_q \cap L_q^{(k)}|}{|R_q|} \tag{6}$$

We also report **Mean Average Precision (MAP)**, **Query Time** (wall-clock, vs. brute-force baseline), and **Approximation Quality** (ratio of retrieved vs. true nearest neighbor distance). Metrics are evaluated at $k \in \{5, 10, 20\}$.

4.3 Baselines and Ablations

The primary baseline is exhaustive brute-force search over GNN embeddings (exact nearest neighbors, no LSH), isolating the cost of LSH approximation from embedding quality. Untrained (random-weight) GNN embeddings serve as a sanity-check baseline.

Table 1: Systems compared in the evaluation.

System	Encoder	Index
Brute-force (exact)	Trained GIN	Exhaustive search
LSH-ANN	Trained GIN	Random projection LSH
Untrained-ANN	Random GIN	Random projection LSH

We ablate over trained vs. untrained embeddings, LSH vs. exact search, varying k, and single vs. multiple hash tables. We also report index construction time, average query latency, and memory usage per hash table.

5 Training

The GIN embedding network is trained end-to-end using triplet loss [4]:

$$\mathcal{L} = \sum_i \max\big(0,\ \|z_{Q_i} - z_{P_i}\|^2 - \|z_{Q_i} - z_{N_i}\|^2 + \alpha\big) \tag{7}$$

where Q_i is an anchor query, P_i is a positive (similar) graph, N_i is a negative (dissimilar) graph, and α is a margin hyperparameter.

5.1 Triplet Construction and the GED Oracle

Triplet construction requires a similarity oracle to distinguish positive from negative graphs. We use approximate GED via the beam search method of Neuhaus et al. (2006), which provides bounded-error approximations in polynomial time.

We explicitly separate the training oracle from evaluation ground truth:

- **Training labels:** Beam search GED ($B = 5$, approximate) used only for ranking graph pairs as similar or dissimilar. The margin $\alpha = 1.0$ is set conservatively to tolerate label noise.
- **Evaluation ground truth:** Exact GED via GEDLib for small graphs (≤ 30 nodes); beam search GED with $B = 20$ for larger datasets, kept entirely separate from the training oracle.

This separation is critical: training and evaluating on the same approximate oracle measures self-consistency, not true retrieval quality. Using exact GED for evaluation wherever feasible gives a meaningful upper bound on true retrieval performance.

6 Related Work

Early approaches to graph retrieval relied on exact similarity measures such as GED and MCS, which are NP-hard and infeasible at scale. Graph Neural Networks introduced a scalable alternative for learning structural representations; among these, GIN is provably as expressive as the Weisfeiler-Leman test [5], making it a strong backbone for structure-sensitive embeddings.

Contrastive and metric learning approaches — including triplet loss [4] — have been applied to graph embeddings to learn spaces where similar graphs cluster together, directly supporting nearest neighbor retrieval. SimGNN [1] is a representative neural approach to fast graph similarity computation.

Approximate Nearest Neighbor methods, including LSH [3] and graph-based indices like HNSW (as in FAISS), enable sublinear-time retrieval over large embedding corpora with minimal accuracy loss. Our work combines GNN-based graph embeddings with LSH-based ANN retrieval under a unified pipeline evaluated on established graph benchmarks.

Acknowledgments

The authors thank Prof. Anirban Dasgupta of the Department of Computer Science and Engineering, IIT Gandhinagar, for proposing this problem and for his guidance throughout the project.

References

[1] Yunsheng Bai, Hao Ding, Song Bian, Ting Chen, Yizhou Sun, and Wei Wang. 2019. SimGNN: A Neural Network Approach to Fast Graph Similarity Computation. In *Proceedings of the 12th ACM International Conference on Web Search and Data Mining (WSDM)*. 31–39.

[2] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-Sensitive Hashing Scheme Based on p-Stable Distributions. In *Proceedings of the 20th Annual Symposium on Computational Geometry (SoCG)*. 253–262.

[3] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. 604–613.

[4] Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. FaceNet: A Unified Embedding for Face Recognition and Clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 815–823.

[5] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.